Computer Organization and Assembly Language

Week 05 — Chapter 2 continued
Size Mismatch, Register Indirect Addressing and the Seven Addressing Modes

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 05 — Lectures 09 and 10

Outline

1. Quick Review
2. Size Mismatch Errors
3. Register Indirect Addressing
4. Register + Offset Addressing
5. Segment Association
6. Address Wraparound
7. Addressing Modes Summary

Quick Review: What Was Done Last Week?

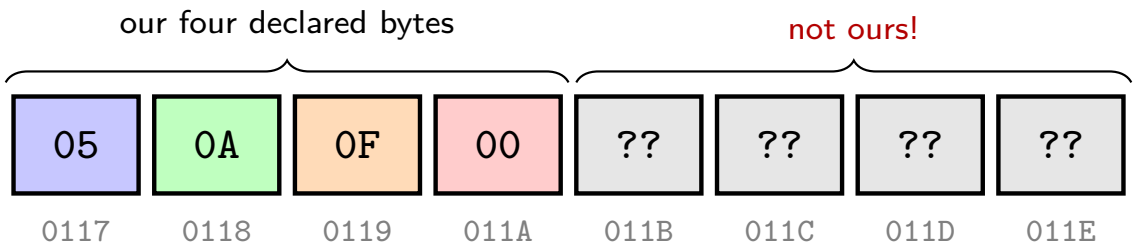
- **Immediate operand** — the value is stored inside the instruction itself. A constant can never be a destination.
- **Data declaration**: `db` reserves 1 byte, `dw` reserves 2 bytes. (`dd` = 4 bytes, `dq` = 8 bytes.)
- **Labels** — a name for an address. The assembler replaces `num1` with a number such as `0117`.
- **Direct addressing** — the address is fixed and written inside the instruction: `A1 17 00` = `mov ax,[num1]`.
- Word variables are **2 apart**, so we wrote `num1+2`, `num1+4`, `num1+6`.
- Memory can be a destination (`add [num1+6], ax`), but memory to memory is illegal. At most **one bracketed operand** per instruction.

Where today starts

Direct addressing fixes the address inside the instruction. Today we learn how to **change the address while the program runs** — which is what makes loops and arrays possible.

What If We Change `dw` to `db`?

- Take the last program and change only the declaration: `num1: db 5, 10, 15, 0`
- The program **still assembles** and **still runs** — no error message at all.
- But the answers are wrong. Why? The declarations now take **1 byte** each, while the instructions still read **2 bytes** each.



- `mov ax,[num1]` reads **two** bytes `05 0A` → AX = `0A05`: two of our numbers glued into one.
- `mov bx,[num1+2]` reads `0F 00` → `000F`: the 15 with the 0 of the next variable attached.
- `[num1+4]` reaches `011B` — outside our data. That value is junk.

Syntax Error vs Logic Error

Syntax error

An instruction the processor *cannot execute*.

The assembler **stops you**.

Example: `mov ax, bl`

Logic error

A legal instruction that does the *wrong thing*.

The assembler **says nothing**.

Example: declaring `db` but reading words

- The assembler lets the programmer do **everything that the processor can possibly do**. It only blocks instructions the CPU cannot execute.
- The assembler and the debugger both did exactly what we asked — we asked for the wrong thing.
- **The processor does not match declarations to accesses.** Keeping them synchronised is the **programmer's** responsibility.

Assembly language gives a lot of power — and power comes with responsibility.
It is not a *difficult* task, it is a *responsible* one.

Example 2.5 — Doing It Correctly With Bytes

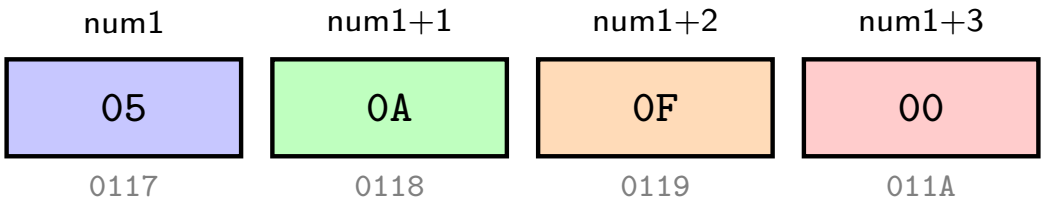
Why this example?

It shows the **right** way to work with byte data: if you declare with db, you must use **byte registers** and **byte offsets** (+1, not +2).

```
; a program to add three numbers using byte variables
[org 0x0100]
    mov al, [num1] ; load first number in al
    mov bl, [num1+1] ; load second number in bl
    add al, bl ; accumulate sum in al
    mov bl, [num1+2] ; load third number in bl
    add al, bl ; accumulate sum in al
    mov [num1+3], al ; store sum at num1+3
    mov ax, 0x4c00 ; terminate program
    int 0x21

num1: db 5, 10, 15, 0
```

Example 2.5 — What Changed and Why



- The number is read into **AL**, a byte register, because the memory location read is also byte sized.
- The second number is now at `num1+1` instead of `num1+2`, because the offsets are now in **bytes**.
- The result is stored at `num1+3` — the fourth byte we declared.
- In the debugger: AL takes the correct values and the sum is stored correctly. **No alignment error this time.**

The rule

Declared as a **byte** → access as a byte. Declared as a **word** → access as a word.

How Does the Processor Know the Size?

- Memory has **no fixed size** — a location can be read as a byte or as a word. So the size comes from the **register**:

`mov ax, [num1]`	AX is 16-bit ⇒ a **word** operation (2 bytes read)
`mov al, [num1]`	AL is 8-bit ⇒ a **byte** operation (1 byte read)
`mov ax, bl`	AX says word, BL says byte ⇒ **ILLEGAL**

The bag analogy

A 5 kg bag cannot fit inside a 1 kg bag — and according to Intel, a 1 kg bag cannot fit inside a 5 kg bag either. **The two operands must match in size.**

- Note the difference: `mov [num1], [num2]` is illegal for a *different* reason — memory-to-memory moves are never allowed, whatever the size.

Ambiguous Instructions

```
mov [num1], 5 ; legal, but AMBIGUOUS
```

- num1 can be treated as a byte or a word.
- 5 can also be a byte or a word.
- **No register is involved**, so the assembler has nothing to guess the size from.
- Remember: memory is a linear array of bytes and a label is just an address in it. A label has no size.

Tell the assembler your intent

```
mov byte [num1], 5 ; write ONE byte
mov word [num1], 5 ; write TWO bytes
```

The Problem With Direct Addressing

- So far we have added **three** numbers. Now imagine **100** numbers.
- With direct addressing that costs about **200 instructions** — one load and one add for every number.
- We need a way to do **the same task repeatedly** on data placed in consecutive memory cells.

The key idea

We need a **register that holds the address** of the data — so that we can *change the address* and reach a different cell using the **same instruction**.

Direct
address fixed *inside* the instruction
`mov ax, [num1]`

→

Indirect
address kept *in a register*
`mov ax, [bx]`

- Only **four** registers can hold an address in iAPX 88: `BX`, `BP`, `SI`, `DI`.

Example 2.6 — Adding With a Pointer

Why this example?

It introduces the pointer idea in its simplest form: put an address in BX, read through [bx], then **move the pointer** to reach the next number.

```
; a program to add three numbers using indirect addressing
[org 0x0100]
    mov bx, num1 ; point bx to first number (NO brackets!)
    mov ax, [bx] ; load first number in ax
    add bx, 2 ; advance bx to second number
    add ax, [bx] ; add second number to ax
    add bx, 2 ; advance bx to third number
    add ax, [bx] ; add third number to ax
    add bx, 2 ; advance bx to result
    mov [bx], ax ; store sum at num1+6
    mov ax, 0x4c00
    int 0x21

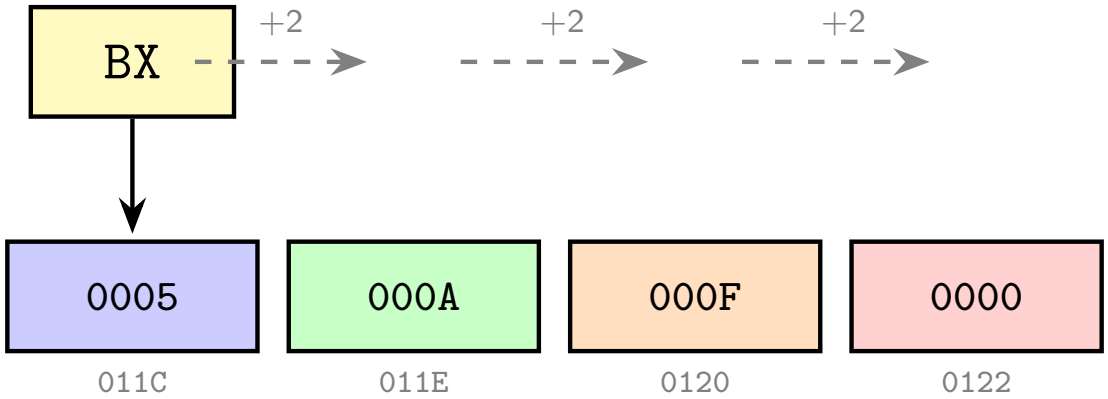
num1: dw 5, 10, 15, 0
```

Brackets or No Brackets? The Most Important Line

`mov bx, num1`	loads the **address** `011C` into BX
`mov bx, [num1]`	would load the **contents** 0005 into BX

- `mov bx, num1` → no brackets → BX gets the **address**. In the debugger this appears as `mov bx, 011C` — just a constant being moved.
- `mov ax, [bx]` → read as: *"move into AX the contents of the memory location whose address is in BX."*
- Brackets may only be placed around **BX, BP, SI, DI** on the iAPX 88. (The 80386 allows more registers.)
- `add bx, 2` → we add **2** because the data are *words*. BX now holds `011E`, the address of the next word.

Example 2.6 — The Pointer Walking Through Memory



- `num1` moved to `011C` because the code became longer — the assembler handled it for us.
- The **same instruction** `add ax, [bx]` reads a different number each time, because BX changed.
- We can also **subtract** from BX to move backwards. The address is now under **full program control** — this is exactly what we needed.

Example 2.7 — A Real Loop Over Ten Numbers

Why this example?

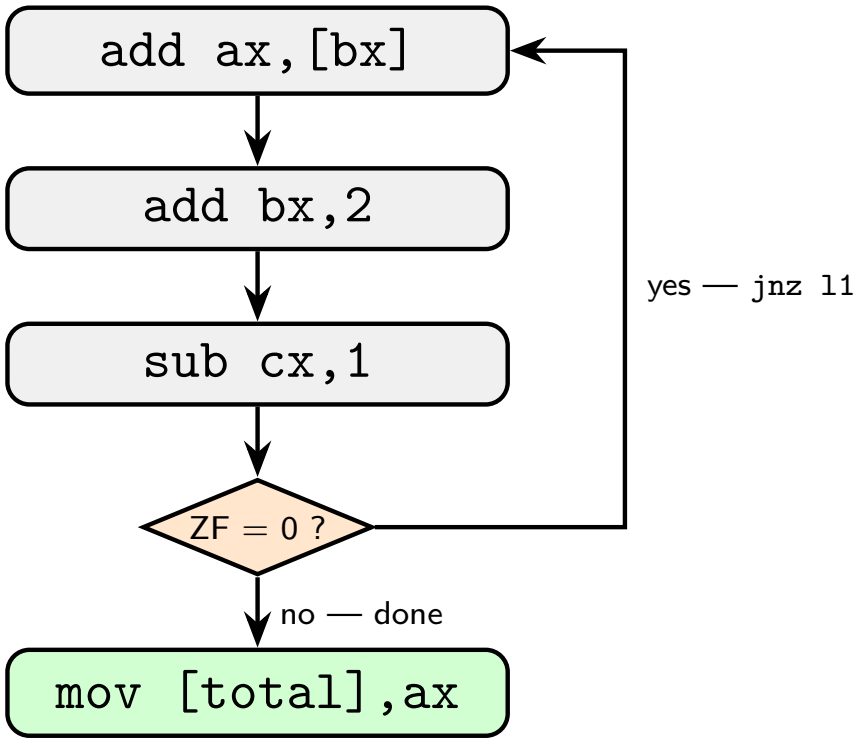
Moving the pointer was the first half. Now we add the second half: a way to **repeat** an instruction and a way to **know when to stop**. This is the first true loop of the course.

```
; a program to add ten numbers
[org 0x0100]
    mov bx, num1 ; point bx to first number
    mov cx, 10 ; load count of numbers in cx
    mov ax, 0 ; initialize sum to zero
l1: add ax, [bx] ; add number to ax
    add bx, 2 ; advance bx to next number
    sub cx, 1 ; numbers to be added reduced
    jnz l1 ; if numbers remain, add next
    mov [total], ax ; write back sum in memory
    mov ax, 0x4c00
    int 0x21

num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0
```

The Three Registers Have Three Different Jobs

BX	CX	AX
the changing **address**	the **counter**	accumulates the **result**



Understanding `JNZ` and the Zero Flag

- **Code labels.** `l1:` is a label on *code*, not data. It remembers the address of the following instruction. The assembler does not distinguish code labels from data labels — that is the programmer's job.
- **SUB** is the counterpart of ADD, with the same rules.
- **JNZ** = "**jump if not zero**". Read it as: *jump to `l1` if the zero flag is not set.*

Zero flag reminder

ZF is set if the last *mathematical or logical* operation produced a zero in its destination.

`mov ax, 0` does **not** set it — MOV is not a mathematical instruction. `sub cx, 1` does.

- So when CX finally becomes 0, ZF is set, the jump is *not* taken, and the loop ends.
- In the debugger: the subtract clears ZF nine times and sets it on the tenth.

The Conditional Jump

- `JNZ` belongs to the **program control** group and is a **conditional jump**:
  - condition true → go to the given label;
  - condition false → continue with the next instruction.
- The processor loads the label's address into `IP` and resumes from there. **The jump breaks the program flow.**
- The destination must *always* be given — just as we say "go *to* that house", never simply "go".
- Warning: the processor jumps blindly. If the label contains data instead of code, it will happily execute the data.

Why this instruction matters

The conditional jump is what gives the processor its **decision-making capability**. Every `if`, `while` and `for` in every language is built on it. Some processors call it *branch* — a more logical name; Intel chose *jump*.

Based and Indexed — Two Names for the Same Idea

- When a **register** is used to reference memory, the access is called **register indirect**.

Register used	Letter stands for	Name of the method
`BX`, `BP`	**B**ase	**Based** addressing
`SI`, `DI`	**I**ndex	**Indexed** addressing

- They have the **same functionality**, with minor differences (which is why the two names exist).
- In Example 2.7 we could have used `SI` or `DI` instead of `BX` — the program would work identically, but we would call it *indexed* addressing.

Example 2.8 — Keeping the Index, Not the Address

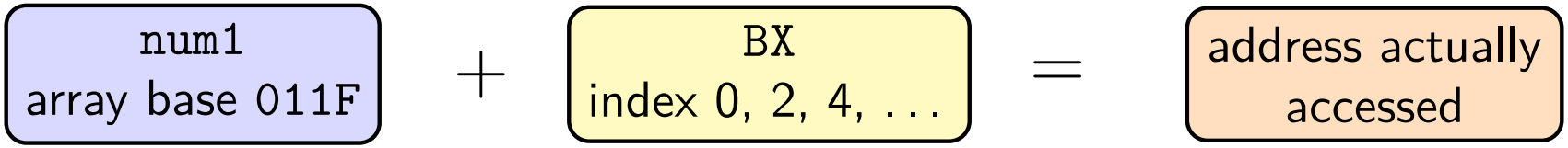
Why this example?

In Example 2.7, BX held the *whole address*. Here BX holds only the **array index**, and the address is formed *at the moment of access*. This lets the same register serve different arrays and be used for index comparison.

```
; add ten numbers using register + offset addressing
[org 0x0100]
    mov bx, 0 ; initialize array INDEX to zero
    mov cx, 10 ; load count of numbers in cx
    mov ax, 0 ; initialize sum to zero
l1: add ax, [num1+bx] ; array base + index, added at access time
    add bx, 2 ; advance bx to next index
    sub cx, 1
    jnz l1
    mov [total], ax
    mov ax, 0x4c00
    int 0x21

num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0
```

Example 2.8 — How the Address Is Built



- BX starts at **0**, not at the array base.
- In the debugger the instruction appears as `mov ax, [011F+bx]` — the address used is the **sum** of `011F` and whatever BX holds.
- BX still jumps in steps of **2**, because each array element is a word.

Important difference from high-level languages

In C++ you write `a[0]`, `a[1]`, `a[2]` and the compiler scales the index for you. **In assembly we always count in bytes**, so we must multiply by the element size ourselves.

The Effective Address

Definition

Every addressing mode ends by producing one number — the **effective address**. In `[num1+bx]`, neither the base nor the index alone names a cell; only *after the addition* does the processor know which cell to access.



- The effective address is just an offset and is meaningless without a segment.
- Segmentation has been happening in *every* example so far, silently. Every access is relative to a segment base.

Default Segment Association

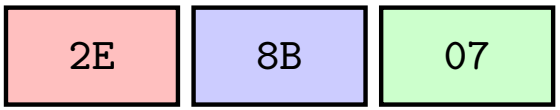
Register used	Default segment	Can it be changed?
`IP`	`CS`	✗ locked, no option
`SP`	`SS`	✗ locked, no option
`BX`, `SI`, `DI`	`DS`	✓ can be overridden
`BP`	`SS`	✓ can be overridden

- `CS` is **tied** to `IP` — code can never be fetched from any other segment.
- For *data* there is relaxation: a **default** association exists, but it can be overridden.
- `BP` defaults to `SS` for a good reason — it will be explained in the chapter on the stack.

The Segment Override Prefix

```
mov ax, [cs:bx] ; use CS with BX -- for THIS instruction only
```

- The next instruction returns to the **default** association.
- The processor places a special byte **before** the instruction, called a prefix — just like prefixes in English.



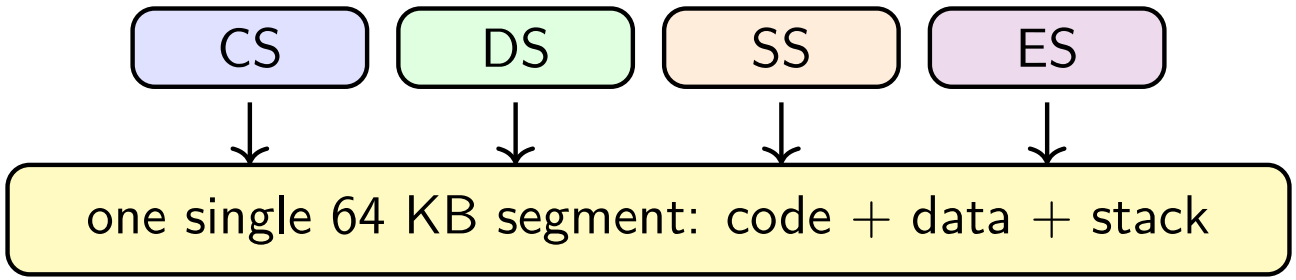
opcode unchanged — only the segment association changed

prefix
(CS override)

- 2E = CS override, 26 = ES override.
- **No prefix is placed** when the default association is used — so default access costs nothing extra.

Why Our Programs Worked Without Any Segment

- In every example so far we never declared or used a segment — yet everything worked.
- The reason: **CS, DS, SS and ES all held the same value**.
  - The value itself does not matter. The fact that they are *equal* is what matters.
- All four segment windows **exactly overlap**, so whichever segment register is used, the same physical memory is reached.



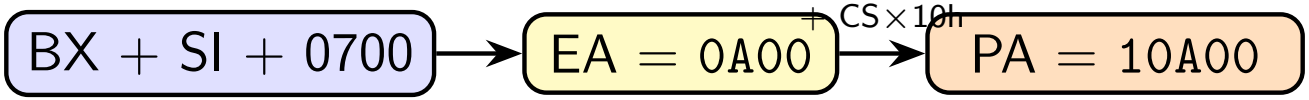
- This is the structure of a **COM file** on the IBM PC — a format defined by **DOS**, not by the processor.
- So the only meaningful thing left for us was the **offset**.

Worked Example: From Registers to a Physical Address

Given: `BX = 0100`, `SI = 0200`, `CS = 1000`, access `[cs:bx+si+0x0700]`

$$\begin{aligned} \text{effective address} &= 0100 + 0200 + 0700 \\ &= \texttt{0A00} \end{aligned}$$

$$\begin{array}{rll} & \texttt{10000} & \leftarrow \text{CS} \times \texttt{10h} \\ + & \texttt{00A00} & \leftarrow \text{effective address} \\ \hline & \texttt{10A00} & \leftarrow \textbf{physical address} \end{array}$$



Two Kinds of Wraparound

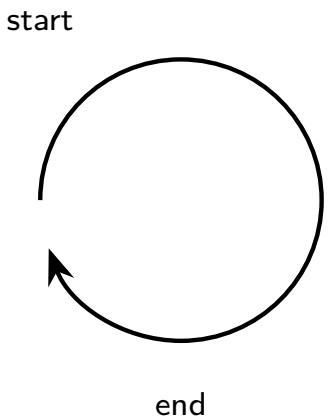
- A wraparound happens when a calculation produces a **carry** that has nowhere to go. The carry is simply **dropped**.

1. Segment wraparound

Happens during **effective address** calculation. Going past the end of a 64 KB segment brings you back to its **start**.

2. Physical wraparound

Happens during **physical address** calculation. Going past the top of the 1 MB brings you back to **address 0**.



Like a circle: an arc at 370°
is the same as an arc at 10°.

Wraparound — Two Worked Examples

Segment wraparound

`BX = 9100, DS = 1500, access [bx+0x7000]`

$$9100 + 7000 = \underbrace{1}_{\text{dropped}}\ 0100 \ \Rightarrow\ \text{EA} = 0100$$

Physical address $= 15000 + 0100 = 15100$.
We tried to cross the segment boundary and were **pushed back to the start**.

Physical wraparound

`BX = 0100, DS = FFF0, access [bx+0x0100]`

$$\text{EA} = 0200, \qquad \texttt{FFF00} + 00200 = \underbrace{1}_{\text{dropped}}\ 00100$$

100100 is a **21-bit** answer and cannot travel on a 20-bit address bus. The access actually happens at 00100 — the very start of memory.

The General Form

```
[ base   +   index   +   offset ]
```

- **base** = `BX` or `BP` **index** = `SI` or `DI` **offset** = a constant
- Skip **any one, any two, or none** of the three parts and you get a legal memory access. That gives the **seven addressing modes** of the iAPX 88.

Remember

Immediate is not an addressing mode — it is an *operand type*. Operands can be **immediate**, **register** or **memory**. Only a memory operand uses one of the seven modes.

Common Mistakes When Forming an Address

Written		**Why**
`[bl]` or `[bh]`	✗	An address is 16-bit; half a register cannot hold it
`[bx-si]`	✗	Only **addition** is allowed
`[bx+bp]`	✗	Two **base** registers
`[si+di]`	✗	Two **index** registers
`[bx+si]`	✓	One base + one index

- One operand must be a **base** and the other an **index** — and *that* is the reason the two groups are named differently.
- **The default segment follows the base register**, not the index register.

The Seven Addressing Modes

Mode	Example	Default seg.
1. Direct	`mov [1234], ax`	DS
2. Based register indirect	`mov [bx], ax`	DS
	`mov [bp], al`	SS
3. Indexed register indirect	`mov [si], ax`	DS
	`mov [di], ax`	DS
4. Based + offset	`mov [bx+300], ax`	DS
	`mov [bp+300], ax`	SS
5. Indexed + offset	`mov [si+300], ax`	DS
	`mov [di+300], al`	DS
6. Base + index	`mov [bx+si], ax`	DS
	`mov [bp+di], al`	SS
7. Base + index + offset	`mov [bx+si+300], ax`	DS

- Notice: whenever `BP` is the base, the default segment becomes **SS**.

Reading the Modes One by One

- **Direct** — a fixed offset in brackets. `mov [1234], ax` stores AX in *two* bytes starting at `1234`; `mov [1234], al` stores *one* byte.
- **Based / Indexed register indirect** — the address comes from the register, so it can change while the program runs.
- **Based / Indexed** + **offset** — register value + a constant. Useful when the constant is the **array base** and the register is the **index** (Example 2.8).
- **Base** + **index** — two registers added. Natural fit for a **two-dimensional array**: one dimension in the base register, the other in the index register.
- **Base** + **index** + **offset** — the most complex form and relatively rare. Array base as the constant, the two dimensions in the two registers; all the address arithmetic is then **delegated to the processor**.

Summary

- **Size mismatch** is a *logic* error, not a syntax error. The assembler will not warn you — declare with `db`, access with byte registers and `+1` offsets.
- The **register** tells the processor the operation size. `mov ax, bl` is illegal; `mov [num1], 5` is ambiguous and needs `byte` or `word`.
- **Register indirect**: put the address in `BX`, `BP`, `SI` or `DI` and read through `[bx]`. Changing the register reaches a new cell with the *same* instruction.
- `mov bx, num1` loads the **address**; `mov bx, [num1]` loads the **contents**.
- **Loop** = pointer + counter + conditional jump. `sub cx,1` then `jnz l1`.
- **Register + offset**: `[num1+bx]` — base in the instruction, index in the register.
- Every mode produces an **effective address**, which is only an offset; the segment completes it.

$$\text{Physical} = (\text{Segment} \times 10\text{h}) + \text{EA}$$

- General form `[base + index + offset]` → **seven addressing modes**.

Thank You

Any Questions?

"Assembly language is not a difficult task, it is a responsible one.
The processor will do exactly what you say — never what you meant."

Next: Chapter 3: Branching...